A38 symbolic benchmark

Task: print `A38_Symbolic_Benchmark_Spec.txt` exactly. That text states the Standard Model with general relativity completely: the gauge group, the matter fields and their charges, the action, the quantum rule, and 28 declared inputs rounded to at most five significant figures.

Run: `python3 A38_Symbolic_Benchmark.py`. It uses only the Python standard library. It compresses the target with raw LZMA2, writes the counted program `A38_Symbolic_Benchmark_Program.py`, runs that program, checks its output against the target by SHA-256 (exact match), and writes `A38_Symbolic_Benchmark_Results.json`.

Result (Python 3.13, CPython):

What | Bytes | Bits
The statement as plain text | 2,549 | 20,392
The statement compressed (raw LZMA2) | 1,416 | 11,328
The self-contained program that prints it | 1,900 | 15,200

What it shows: a complete statement of the Standard Model with general relativity, laws and constants together, prints from a program of 15,200 bits in a declared language, and its compressed core is 11,328 bits. That is an upper bound for this task in this language. A shorter program may exist.

What it does not do: it computes no predictions. A program that derives observable predictions from these laws is the next, predictive task, and it has to be compared with rivals doing the same task in the same language. The inputs are rounded values near current central values, declared for the count, and not a new fit.

The other A38 supplements still run and reproduce their shipped outputs: `A38_Coupling_Code_Benchmark.py` reproduces

A38_Coupling_Code_Results.json exactly, and A38_physics_spec.c compiles and prints a CKM matrix that is unitary to within 2.2e-16. Their scope is stated in A38_C_BINARY_BIT_COUNT.md.